

Build web apps faster with **Cursor**

Cursor is a code editor built from the ground up for AI-assisted development. It understands your entire project, edits multiple files at once, and acts as a development partner — not just an autocomplete tool.

THE BASICS

What is Cursor?

Cursor is a fork of VS Code redesigned around AI. It looks familiar if you've used VS Code, but the AI isn't a plugin bolted on the side — it's woven into every part of the editing experience.

Traditional IDE + Copilot

- AI suggests the next line as you type
- Mostly single-file context
- You drive every file change manually
- Chat is a separate sidebar add-on

Cursor (AI-first)

- ✓ Agent mode edits across your whole codebase
- ✓ Understands project structure, rules, and docs
- ✓ Runs terminal commands, tests, and multi-step tasks
- ✓ MCP connects to GitHub, browsers, databases, and more

Your value as a developer is in **decision-making**, not typing speed. Cursor amplifies your decisions — it doesn't replace them.

CORE TOOLS

Four ways to work with AI

Each mode serves a different job. Learning when to reach for each one is the single biggest skill for getting great results.

Cmd+L

Chat

EXPLORE & ASK

Ask questions about your codebase, get explanations, and make focused edits to a single file. Great for understanding unfamiliar code before you change it.

Cmd+K

Inline Edit

QUICK CHANGES

Select code, describe what you want, and get a targeted rewrite in place. Perfect for refactoring a function, fixing a bug, or tweaking a component.

Cmd+I

Agent

BUILD FEATURES

The powerhouse mode. Agent reads your project, creates and edits multiple files, runs commands, and implements entire features — like adding auth, a new API route, or a dashboard page.

Tab

Autocomplete

STAY IN FLOW

Cursor predicts your next edit — sometimes whole blocks — as you type. Accept with Tab to move fast through routine code without breaking focus.

REAL-WORLD USE

What Cursor can do for web app developers

From a blank HTML file to a deployed Next.js app with auth, payments, and AI features — here's what developers actually use Cursor for.



Explore unfamiliar codebases

Onboard to a new repo fast. Ask "how does authentication work?" or "where are API routes defined?" and get answers grounded in your actual files.



Build features end-to-end

Describe a feature — a quiz page, a settings screen, a Stripe checkout flow — and Agent creates the components, hooks, API routes, and types together.



Design → code

Paste a screenshot or sketch of a UI and ask Cursor to implement it with your existing component library (Tailwind, shadcn/ui, etc.).



Test-driven development

Write a failing test first, then let Agent implement the code to make it pass. Cursor can run your test suite and iterate until green.



Architecture diagrams

Generate Mermaid diagrams for data flows, component trees, and system architecture — useful for planning and documentation.



MCP integrations

Connect external tools via Model Context Protocol: browse pages with Playwright, manage GitHub PRs, query docs, and interact with databases — all from the editor.



Debug and fix issues

Paste an error message or failing CI log. Cursor traces the stack, reads relevant files, and proposes a fix — often across several files at once.



Ship to production

Configure deployment, environment variables, database migrations, and CI/CD. Cursor understands the full stack from local dev to Vercel production.



HOW TO WORK

A workflow that actually works

The developers who get the most from Cursor don't just prompt harder — they structure their work differently.

Explore → Plan → Build

1

Explore

Use Chat and @ symbols to understand the codebase before writing a single line. Don't jump straight to "build me X."

2

Plan

Ask Cursor to plan (not build). Or use ChatGPT/Claude for brainstorming, then bring the plan to Cursor for implementation.

3

Build

Switch to Agent with a clear, scoped task. Review diffs, run tests, commit. Repeat in small increments.

Hierarchy of Leverage

Rules — encode team standards once, apply everywhere

Context — curate what the AI sees (@files, AGENTS.md)

Prompts — clear, scoped instructions per task

Model — pick the right model, but it's the smallest lever

Invest in rules and context before chasing better prompts or bigger models.

Context navigation with @

@file

One specific file

@docs

Library documentation

@folder

A directory

@web

Live web search

@codebase

Search whole project

@git

Recent changes

BEST PRACTICES

Tips for getting great results

Cursor is powerful, but it's not magic. These habits separate smooth sessions from frustrating ones.

Commit often with Git

Cursor's checkpoint system is not a replacement for version control. Commit after every working milestone so you can always roll back.

Try five times, then rephrase

If a prompt doesn't work, don't give up — rephrase it, add context with @, or break the task into smaller steps.

Curate context, don't dump everything

More context isn't always better. Use @ to attach only the files that matter. Start a new chat when context gets long.

Start without rules, add when needed

Create `.cursor/rules/*.mdc` files when you notice repeated mistakes — not before you've written any code.

Plan in one tool, build in another

Use ChatGPT or Claude for brainstorming and architecture. Bring a clear plan to Cursor Agent for implementation.

Work in small increments

One feature at a time. Write a test, implement, verify, commit. Large vague prompts lead to large vague bugs.

Review every diff

You're still the engineer. Read what Agent changed before accepting. Catch hallucinated imports, wrong patterns, and security issues early.

Use project rules for conventions

Encode Next.js patterns, API standards, and testing conventions in rules so the AI follows your team's style automatically.

READY TO GO

Start building today

Install Cursor, open a project (or start from scratch), and try the four modes on something small. A landing page, a quiz component, a API route — anything that lets you feel the workflow before tackling something bigger.

Get Cursor →

Built for the O'Reilly live training Building Production Applications with Cursor AI IDE. [Official docs](#) · [Rule examples](#)